BetaDanish

An R Package for Survival, Reliability, and Lifetime Data Analysis

User Manual

Version 0.2.0

Bilal Ahmad | Muhammad Yameen Danish

Allama Iqbal Open University, Islamabad, Pakistan

Reference: Ahmad, B., & Danish, M. Y. (2025).

Package website:
https://bilal-aiou.github.io/BetaDanish/

GitHub repository:
https://github.com/bilal-aiou/BetaDanish

TABLE OF CONTENTS

Part I — Background and Introduction 5
1. What is BetaDanish? 5
2. Theoretical Background 5
2.1 The Danish Baseline 5
2.2 The Beta-Danish Construction 5
2.3 Why a New Distribution? 5
3. What This Package Lets You Do 6
Part II — Installation and First Steps 7
4. Installation 7
4.1 Installing from CRAN (after CRAN acceptance) 7
4.2 Installing from GitHub (development version) 7
4.3 Installing from a Local Tarball 7
4.4 Optional Suggested Packages 7
5. Loading the Package 7
6. Verifying the Installation 7
Part III — Complete Function Reference 9
7. All Exported Functions at a Glance 9
7.1 Core distribution functions 9
7.2 Maximum likelihood estimation 9
7.3 Bayesian inference 9
7.4 Regression and advanced models 9
7.5 Competing-risks utilities 10
7.6 Structural properties 10
7.7 Diagnostics and visualization 10
7.8 Reports and pipelines 10
7.9 Data utilities and simulation 10
7.10 S3 methods (used automatically by generic functions) 11
8. Built-in Datasets 11
Part IV — Command Reference with Detailed Examples 12
9. Distribution Functions 12
9.1 dbetadanish() — Density 12
9.2 pbetadanish() — CDF and survival 12

9.3 qbetadanish() — Quantile function 13
9.4 rbetadanish() — Random sampling 13
9.5 hbetadanish() — Hazard rate 13
10. Maximum Likelihood Estimation 14
10.1 fit_betadanish() — Fit the full or submodel 14
10.2 compare_models() — LRT for the submodel 15
10.3 compare_distributions() — Cross-distribution comparison 15
10.4 gof_betadanish() — Goodness-of-fit summary 15
11. Bayesian Inference 15
11.1 bayes_betadanish() — MCMC sampling 15
12. AFT Regression 16
12.1 fit_bd_aft() — Accelerated Failure Time model 16
13. Cure Models 17
13.1 fit_bd_cure() — Mixture and promotion-time cure 17
13.2 simulate_bd_cure_data() — Simulate cure data 17
14. Competing Risks 18
14.1 fit_bd_competing() — Joint competing-risks model 18
14.2 cif_betadanish() — Fitted cumulative incidence 18
14.3 cif_compare() — CIF vs Aalen-Johansen with Gray's test 18
15. Structural Properties 18
15.1 bd_entropy_shannon() — Differential Shannon entropy 18
15.2 bd_order_stat_pdf() — r-th order statistic density 19
16. Diagnostic Plots 19
16.1 plot.betadanish() — Comprehensive diagnostics 19
16.2 plot.bd_aft() and plot.bd_cure() — Cox-Snell residuals 19
17. End-to-End Reports 19
17.1 analyze_betadanish() — One-call pipeline 19
17.2 report_betadanish() — Printable summary 20
18. Data Utilities and Simulation 20
18.1 read_survival_data() — CSV/Excel reader 20
18.2 simulate_bd_data() — Generate survival data 20
18.3 simulate_bd_cure_data() — Generate cure-model data 20
Part V — Walkthroughs 21
19. A Complete Worked Example 21

Step 1: Load and inspect the data 21
Step 2: Plot the empirical Kaplan-Meier 21
Step 3: Fit the four-parameter Beta-Danish model 21
Step 4: Fit the three-parameter ED submodel 21
Step 5: Likelihood ratio test (full vs submodel) 21
Step 6: Compare against standard distributions 21
Step 7: Diagnostic plots 21
Step 8: Optional Bayesian inference 22
20. Quick Reference Card 22
Part VI — Appendices 23
Appendix A — Getting Help 23
Appendix B — Reporting Bugs and Feature Requests 23
Appendix C — Citation 23
Appendix D — Session Information 23
Appendix E — License 23
Appendix F — Authors and Acknowledgements 23

Part I — Background and Introduction

1. What is BetaDanish?

BetaDanish is an R package that implements the four-parameter Beta-Danish distribution and its three-parameter Exponentiated Danish (ED) submodel for survival, reliability, and lifetime data analysis. The package was developed alongside the doctoral research of Bilal Ahmad under the supervision of Dr. Muhammad Yameen Danish at Allama Iqbal Open University, Islamabad.

The Beta-Danish distribution offers a flexible alternative to classical lifetime models such as the Weibull, gamma, and log-normal. It can accommodate monotonically increasing, monotonically decreasing, unimodal, and bathtub-shaped hazard rates — all within a single parametric family. This flexibility makes it particularly suitable for medical survival data, industrial reliability data, and any setting in which the hazard shape is not known a priori.

2. Theoretical Background

2.1 The Danish Baseline

The Beta-Danish distribution is built upon the Danish distribution as its baseline. The Danish distribution has CDF and PDF:

```
G(z; c, k) = ( k*z / (1 + k*z) )^c
g(z; c, k) = c * k * (k*z)^(c-1) * (1 + k*z)^(-(c+1))
```

where c > 0 is a shape parameter and k > 0 is a scale parameter, defined for z > 0.

2.2 The Beta-Danish Construction

Applying the beta-generated family of Eugene, Lee, and Famoye (2002) to the Danish baseline gives the Beta-Danish CDF:

```
F_Z(z; a, b, c, k) = I_{G(z; c, k)}(a, b)
                   = I_{ (k*z/(1+k*z))^c }(a, b)
```

where I_x(a, b) is the regularized incomplete beta function and a, b, c, k > 0. The corresponding probability density function (equation 5.13 of the underlying thesis) is:

```
                  c*k          (k*z)^(c*a - 1)              k*z   c    b-1
f_Z(z; a,b,c,k) = ----- * ------------------------ * [1 - (-----) ]
                  B(a,b)     (1 + k*z)^(c*a + 1)           1+k*z
```

Setting a = 1 yields the three-parameter Exponentiated Danish (ED) submodel, which is used throughout the empirical case studies in the thesis.

2.3 Why a New Distribution?

The Beta-Danish distribution was developed to address three recurring needs in lifetime data analysis:

- Flexibility of hazard shape — the four-parameter form accommodates increasing, decreasing, unimodal, and bathtub-shaped hazards within one family, eliminating the need for ad-hoc model selection between Weibull, log-normal, and log-logistic alternatives.

- Closed-form structural properties — moments, the survival and hazard functions, the quantile function, and Shannon entropy all admit tractable expressions, making both mathematical analysis and computational implementation efficient.
- Compatibility with modern survival modelling — the distribution serves as a baseline for AFT regression, mixture and promotion-time cure models, and competing-risks analysis, all of which are implemented in this package.

3. What This Package Lets You Do

The BetaDanish package provides a comprehensive toolkit covering eight broad areas of survival and reliability analysis:

- Compute the density, distribution, quantile, hazard, and survival functions of the Beta-Danish family
- Generate random samples for simulation studies
- Fit Beta-Danish models by maximum likelihood to complete or right-censored data
- Fit Beta-Danish models by Bayesian MCMC with vague priors
- Fit Accelerated Failure Time (AFT) regression models with covariates
- Fit mixture and promotion-time (non-mixture) cure models
- Fit and compare competing-risks models against the nonparametric Aalen-Johansen estimator with Gray's test
- Produce diagnostic plots, goodness-of-fit summaries, and structural-property calculations (entropy, order statistics)

Part II — Installation and First Steps

4. Installation

4.1 Installing from CRAN (after CRAN acceptance)

Once the package is accepted by CRAN, install it using the standard install.packages() function:

```
install.packages("BetaDanish")
```

4.2 Installing from GitHub (development version)

To install the latest development version directly from GitHub:

```
# install.packages("devtools")    # only if devtools not installed
devtools::install_github("bilal-aiou/BetaDanish")
```

4.3 Installing from a Local Tarball

If you have the source tarball (e.g., BetaDanish_0.2.0.tar.gz):

```
install.packages("path/to/BetaDanish_0.2.0.tar.gz",
                 repos = NULL, type = "source")
```

4.4 Optional Suggested Packages

Several advanced features depend on optional packages declared in Suggests:

```
install.packages(c("MCMCpack",   # Bayesian estimation
                   "coda",       # MCMC diagnostics
                   "cmprsk",     # competing-risks comparison
                   "flexsurv",   # comparison with standard distributions
                   "MASS"))      # multivariate normal sampling
```

5. Loading the Package

Once installed, load the package into your R session:

```
library(BetaDanish)
```

A startup message confirms the version. You can now access all exported functions.

6. Verifying the Installation

To confirm that everything is working, run a minimal end-to-end example:

```
library(BetaDanish)
set.seed(123)
dat <- simulate_bd_data(n = 100, a = 1.5, b = 2, c = 3, k = 0.5)
head(dat)
fit <- fit_betadanish(survival::Surv(time, status) ~ 1, data = dat)
summary(fit)
```

If you see a fitted-model summary with parameter estimates, standard errors, log-likelihood, and AIC/BIC, your installation is correct.

Part III — Complete Function Reference

7. All Exported Functions at a Glance

The package exports 22 functions across nine functional categories. The table below lists every exported function in the package and what it does.

7.1 Core distribution functions

Function	Purpose
`dbetadanish()`	Probability density function (PDF) of the Beta-Danish distribution
`pbetadanish()`	Cumulative distribution function (CDF) and survival function
`qbetadanish()`	Quantile function (inverse CDF)
`rbetadanish()`	Random number generation from the Beta-Danish distribution
`hbetadanish()`	Hazard rate function

7.2 Maximum likelihood estimation

Function	Purpose
`fit_betadanish()`	Fit the Beta-Danish or ED submodel to censored data by MLE
`compare_models()`	Likelihood ratio test comparing the full Beta-Danish to the ED submodel
`compare_distributions()`	Compare the Beta-Danish fit against standard distributions (Weibull, gamma, log-normal, exponential)
`gof_betadanish()`	Goodness-of-fit summary (AIC, BIC, AICC, HQIC, KS statistic)

7.3 Bayesian inference

Function	Purpose
`bayes_betadanish()`	Random-walk Metropolis sampling of the posterior distribution

7.4 Regression and advanced models

Function	Purpose
`fit_bd_aft()`	Fit an Accelerated Failure Time (AFT) regression model
`fit_bd_cure()`	Fit a mixture or promotion-time cure model

Function	Purpose
`fit_bd_competing()`	Fit a competing-risks model with cause-specific Beta-Danish marginals

7.5 Competing-risks utilities

Function	Purpose
`cif_betadanish()`	Compute the fitted cumulative incidence function for a given cause
`cif_compare()`	Compare fitted CIFs against the nonparametric Aalen-Johansen estimator and report Gray's test

7.6 Structural properties

Function	Purpose
`bd_entropy_shannon()`	Shannon (differential) entropy via Gauss-Kronrod quadrature
`bd_order_stat_pdf()`	Density of the r-th order statistic from a sample of size n

7.7 Diagnostics and visualization

Function	Purpose
`plot.betadanish()`	Diagnostic plots: survival, hazard, density, P-P, Q-Q, or all
`plot.bd_aft()`	Cox-Snell residual plot for AFT fits
`plot.bd_cure()`	Cox-Snell residual plot for cure model fits

7.8 Reports and pipelines

Function	Purpose
`analyze_betadanish()`	End-to-end analysis pipeline: reads data, fits full and submodel, compares them, plots diagnostics
`report_betadanish()`	Printable report summarizing a fitted model

7.9 Data utilities and simulation

Function	Purpose
`read_survival_data()`	Read survival data from CSV or Excel with automatic NA handling
`simulate_bd_data()`	Simulate complete or right-censored data from the Beta-Danish

Function	Purpose
simulate_bd_cure_data()	Simulate data with a cure fraction from either cure formulation

7.10 S3 methods (used automatically by generic functions)

Function	Purpose
summary()	Summarize any fitted model object
print()	Print any fitted model object
coef()	Extract estimated coefficients
vcov()	Extract the variance-covariance matrix
logLik()	Extract the maximized log-likelihood

8. Built-in Datasets

The package ships with seven curated datasets covering reliability and clinical survival settings:

Function	Purpose
remission	Bladder cancer remission times for 128 patients
carbon_fibres	Breaking stress (in Gba) of 100 carbon fibre specimens
transplant	Bone marrow transplant survival for 91 patients with a treatment covariate
aarset	Aarset device failure times (n = 50, exhibits a classic bathtub hazard)
leukemia	Acute myelogenous leukemia survival (n = 23)
melanoma	Malignant melanoma survival after surgery (n = 205)
brain_cancer	Brain cancer survival data with multiple comorbidity covariates

Load any dataset with the data() function, for example: data(remission)

Part IV — Command Reference with Detailed Examples

9. Distribution Functions

The five core distribution functions follow the standard R d/p/q/r/h naming convention and accept vectorized inputs.

9.1 dbetadanish() — Density

Computes the probability density function (PDF).

Syntax:

```
dbetadanish(x, a, b, c, k, log = FALSE)
```

Arguments:

Argument	Type	Description
x	*numeric*	Vector of values (must be > 0) at which to evaluate the density.
a	*numeric*	Shape parameter (a > 0).
b	*numeric*	Shape parameter (b > 0).
c	*numeric*	Shape parameter (c > 0).
k	*numeric*	Scale parameter (k > 0).
log	*logical*	If TRUE, return the log-density (more numerically stable for likelihood work).

Example:

```
# PDF values at three time points
dbetadanish(c(0.5, 1, 2), a = 1.5, b = 2, c = 3, k = 0.5)

# Plot the density curve
x <- seq(0.01, 5, length.out = 200)
plot(x, dbetadanish(x, a = 1.5, b = 2, c = 3, k = 0.5),
     type = "l", lwd = 2, col = "steelblue",
     xlab = "t", ylab = "f(t)",
     main = "Beta-Danish PDF")
```

9.2 pbetadanish() — CDF and survival

Computes the cumulative distribution function or the survival function depending on the lower.tail flag.

Syntax:

```
pbetadanish(q, a, b, c, k,
            lower.tail = TRUE, log.p = FALSE)
```

Arguments:

Argument	Type	Description
q	*numeric*	Vector of quantiles (must be > 0).
a, b, c, k	*numeric*	Distribution parameters (all > 0).
lower.tail	*logical*	If TRUE, returns F(q) = P(T <= q); if FALSE, returns S(q) = P(T > q).
log.p	*logical*	If TRUE, return log probabilities.

Example:

```
# Probability that T <= 2
pbetadanish(2, a = 1.5, b = 2, c = 3, k = 0.5)

# Survival probability that T > 2
pbetadanish(2, a = 1.5, b = 2, c = 3, k = 0.5, lower.tail = FALSE)
```

9.3 qbetadanish() — Quantile function

Inverse CDF. Returns the time at which a given cumulative probability is reached.

Syntax:

```
qbetadanish(p, a, b, c, k,
            lower.tail = TRUE, log.p = FALSE)
```

Example:

```
# Median (50th percentile) of the distribution
qbetadanish(0.5, a = 1.5, b = 2, c = 3, k = 0.5)

# 5th, 25th, 75th, 95th percentiles
qbetadanish(c(0.05, 0.25, 0.75, 0.95),
            a = 1.5, b = 2, c = 3, k = 0.5)
```

9.4 rbetadanish() — Random sampling

Generates pseudo-random samples from the Beta-Danish distribution by inverse-transform sampling.

Syntax:

```
rbetadanish(n, a, b, c, k)
```

Example:

```
set.seed(42)
x <- rbetadanish(1000, a = 1.5, b = 2, c = 3, k = 0.5)
hist(x, breaks = 40, freq = FALSE,
     main = "Histogram of simulated data",
     xlab = "t")
curve(dbetadanish(x, 1.5, 2, 3, 0.5),
      add = TRUE, col = "red", lwd = 2)
```

9.5 hbetadanish() — Hazard rate

Computes the hazard rate (instantaneous failure rate) h(t) = f(t) / S(t).

Syntax:

```
hbetadanish(x, a, b, c, k, log = FALSE)
```

Example:

```
# Compare bathtub vs increasing hazard shapes
t <- seq(0.01, 5, length.out = 200)
plot(t,  hbetadanish(t, a = 0.5, b = 0.5, c = 2, k = 0.5),
     type = "l", col = "red", lwd = 2,
     xlab = "t", ylab = "h(t)",
     main = "Hazard shapes")
lines(t, hbetadanish(t, a = 2, b = 2, c = 2, k = 0.5),
      col = "blue", lwd = 2)
legend("topright",
       c("bathtub: a=b=0.5", "unimodal: a=b=2"),
       col = c("red", "blue"), lwd = 2)
```

10. Maximum Likelihood Estimation

10.1 fit_betadanish() — Fit the full or submodel

Fits the Beta-Danish distribution (or its ED submodel) by maximum likelihood. Supports right-censored data via survival::Surv.

Syntax:

```
fit_betadanish(formula, data,
               submodel = FALSE,
               n_starts = 10,
               method = "BFGS")
```

Arguments:

Argument	Type	Description
formula	*formula*	Right-hand side ~ 1 for intercept-only. Left-hand side must be Surv(time, status).
data	*data.frame*	Data frame containing the variables in the formula.
submodel	*logical*	If TRUE, fit the 3-parameter ED submodel (a fixed at 1).
n_starts	*integer*	Number of random starts for the optimizer (default 10). More = more robust to local optima.
method	*character*	Optimization method passed to maxLik (BFGS is recommended).

Returns:

An object of S3 class "betadanish" containing estimates, the variance-covariance matrix on the natural scale, the log-likelihood, and convergence diagnostics.

Example:

```
data(remission)
fit <- fit_betadanish(survival::Surv(time, status) ~ 1,
                      data = remission)
summary(fit)
plot(fit, type = "all")
```

10.2 compare_models() — LRT for the submodel

Performs a likelihood ratio test comparing the full 4-parameter model against the 3-parameter ED submodel (H0: a = 1).

Syntax:

```
compare_models(full_model, sub_model)
```

Example:

```
data(remission)
fit_full <- fit_betadanish(survival::Surv(time, status) ~ 1,
                           data = remission)
fit_sub  <- fit_betadanish(survival::Surv(time, status) ~ 1,
                           data = remission,
                           submodel = TRUE)
compare_models(fit_full, fit_sub)
```

10.3 compare_distributions() — Cross-distribution comparison

Compares the Beta-Danish fit against four standard lifetime distributions (Weibull, gamma, log-normal, exponential) by AIC and BIC.

Syntax:

```
compare_distributions(object)
```

Example:

```
data(remission)
fit <- fit_betadanish(survival::Surv(time, status) ~ 1,
                      data = remission)
compare_distributions(fit)
```

10.4 gof_betadanish() — Goodness-of-fit summary

Returns AIC, BIC, AICC, HQIC, and the Kolmogorov-Smirnov goodness-of-fit statistic.

Syntax:

```
gof_betadanish(object)
```

Example:

```
gof_betadanish(fit)
```

11. Bayesian Inference

11.1 bayes_betadanish() — MCMC sampling

Samples from the posterior of the Beta-Danish parameters using a random-walk Metropolis sampler with vague Gamma(0.01, 0.01) priors. Requires the optional MCMCpack and coda packages.

Syntax:

```
bayes_betadanish(time, status = NULL,
                 submodel = TRUE,
                 burnin = 5000, mcmc = 15000,
                 tune = 0.5,
                 theta_init = NULL,
                 seed = NULL, verbose = 0)
```

Arguments:

Argument	Type	Description
`time`	*numeric vector*	Observed times (must be > 0).
`status`	*numeric vector*	Event indicators: 1 = event, 0 = right-censored. If NULL, all observations treated as events.
`submodel`	*logical*	If TRUE, sample from the 3-parameter ED submodel; if FALSE, the full 4-parameter model.
`burnin`	*integer*	Number of burn-in iterations to discard.
`mcmc`	*integer*	Number of post-burn-in samples to retain.
`tune`	*numeric*	Random-walk proposal tuning parameter (larger = bigger steps).
`theta_init`	*numeric*	Optional log-scale starting values.
`seed`	*integer*	Reproducibility seed.

Example:

```
data(remission)
fit_bayes <- bayes_betadanish(
  time     = remission$time,
  status   = remission$status,
  submodel = TRUE,
  burnin   = 2000, mcmc = 5000,
  seed     = 1
)
print(fit_bayes)

# Trace plots and convergence diagnostics
coda::traceplot(fit_bayes$draws)
coda::effectiveSize(fit_bayes$draws)
```

12. AFT Regression

12.1 fit_bd_aft() — Accelerated Failure Time model

Fits an AFT regression model on the Exponentiated Danish kernel. Covariates enter through the scale parameter k via k_i = exp(X_i' delta).

Syntax:

```
fit_bd_aft(formula, data,
           n_starts = 10, method = "BFGS")
```

Example:

```
data(brain_cancer)
fit_aft <- fit_bd_aft(
  survival::Surv(Survtime, Survstatus) ~ Age + Grade + Surgery,
  data = brain_cancer
)
summary(fit_aft)
plot(fit_aft)   # Cox-Snell residual diagnostic
```

13. Cure Models

13.1 fit_bd_cure() — Mixture and promotion-time cure

Fits two cure formulations:

- Mixture cure model: population is split into susceptible and cured fractions via logistic regression on incidence covariates.
- Promotion-time (non-mixture) cure model: cure fraction arises from a latent Poisson process of clonogenic cells with intensity theta(Z) = exp(Z' gamma).

Syntax:

```
fit_bd_cure(formula_aft, formula_cure, data,
            type = c("mixture", "promotion"),
            n_starts = 10, method = "BFGS")
```

Example:

```
data(transplant)
fit_cure <- fit_bd_cure(
  formula_aft  = survival::Surv(time, status) ~ 1,
  formula_cure = ~ group,
  data         = transplant,
  type         = "mixture"
)
summary(fit_cure)
plot(fit_cure)
```

13.2 simulate_bd_cure_data() — Simulate cure data

Generates synthetic data with a known cure fraction for either cure formulation.

```
# Simulate from the mixture cure structure
dat <- simulate_bd_cure_data(n = 300, type = "mixture",
                             a = 1, b = 2, c = 1.5,
                             gamma = c(0.3, 0.7), seed = 1)
head(dat)
```

14. Competing Risks

14.1 fit_bd_competing() — Joint competing-risks model

Fits a parametric competing-risks model assuming independent latent failure times, with each cause-specific marginal following a 4-parameter Beta-Danish distribution.

Syntax:

```
fit_bd_competing(time, cause,
                 n_starts = 5,
                 method = "BFGS")
```

Arguments:

Argument	Type	Description
time	*numeric vector*	Observed times.
cause	*integer vector*	Cause codes: 0 = censored, 1, 2, ..., m = specific event causes.
n_starts	*integer*	Random starts for joint optimization.

Example:

```
set.seed(2026)
n  <- 400
T1 <- rbetadanish(n, a = 1.2, b = 1.5, c = 1.0, k = 0.4)
T2 <- rbetadanish(n, a = 1.0, b = 2.0, c = 1.0, k = 0.2)
C  <- stats::rexp(n, 0.05)
time  <- pmin(T1, T2, C)
cause <- ifelse(time == C, 0L, ifelse(T1 <= T2, 1L, 2L))
fit_cr <- fit_bd_competing(time = time, cause = cause)
summary(fit_cr)
```

14.2 cif_betadanish() — Fitted cumulative incidence

Evaluates the fitted cumulative incidence function (CIF) for a specified cause at given times.

```
tgrid <- seq(0, 30, length.out = 200)
cif1 <- cif_betadanish(fit_cr, tvec = tgrid, cause_idx = 1)
```

14.3 cif_compare() — CIF vs Aalen-Johansen with Gray's test

Overlays the fitted Beta-Danish CIF against the nonparametric Aalen-Johansen estimator and reports Gray's test for CIF equality. Requires the cmprsk package.

```
res <- cif_compare(fit_cr, plot = TRUE)
res$gray_test
```

15. Structural Properties

15.1 bd_entropy_shannon() — Differential Shannon entropy

Computes H(f) = -integral of f(t) log f(t) dt by adaptive Gauss-Kronrod quadrature.

```
# Entropy of a regular configuration
bd_entropy_shannon(a = 1.5, b = 2.5, c = 2, k = 1)
```

15.2 bd_order_stat_pdf() — r-th order statistic density

Returns the density (or log-density) of the r-th order statistic from an i.i.d. sample of size n.

```
tgrid <- seq(0.01, 5, length.out = 200)
# Density of the median in a sample of size 21
fm <- bd_order_stat_pdf(tgrid, r = 11, n = 21,
                        a = 1.5, b = 2.5, c = 2, k = 1)
plot(tgrid, fm, type = "l", lwd = 2,
     main = "Density of the sample median (n = 21)",
     xlab = "t", ylab = "f(11:21)(t)")
```

16. Diagnostic Plots

16.1 plot.betadanish() — Comprehensive diagnostics

Six plot types are available, controlled by the type argument:

Function	Purpose
`type = "survival"`	Kaplan-Meier vs fitted survival curve
`type = "hazard"`	Fitted hazard rate over time
`type = "density"`	Histogram of observed times with fitted density overlay
`type = "pp"`	Probability-probability plot
`type = "qq"`	Quantile-quantile plot
`type = "all"`	All five plots arranged in a 2x3 panel

```
plot(fit, type = "all")
plot(fit, type = "survival")
plot(fit, type = "hazard")
```

16.2 plot.bd_aft() and plot.bd_cure() — Cox-Snell residuals

For AFT and cure fits, plots the Kaplan-Meier estimate of the cumulative hazard of the Cox-Snell residuals against the 45-degree line. A well-fitting model produces a curve close to the diagonal.

```
plot(fit_aft)   # Cox-Snell residuals for an AFT fit
plot(fit_cure)  # Cox-Snell residuals for a cure fit
```

17. End-to-End Reports

17.1 analyze_betadanish() — One-call pipeline

Reads survival data, fits the full and submodel, runs an LRT, computes goodness-of-fit, and produces diagnostic plots — all in one function call.

```
res <- analyze_betadanish(
  file       = "my_survival_data.csv",
  time_col   = "time",
  status_col = "event"
)
```

17.2 report_betadanish() — Printable summary

Returns a print-friendly report of a fitted model.

```
report_betadanish(fit)
```

18. Data Utilities and Simulation

18.1 read_survival_data() — CSV/Excel reader

Reads survival data from a .csv, .xlsx, or .xls file, handles missing values, and produces a clean data frame with standardized column names ready for fit_betadanish().

```
dat <- read_survival_data(
  file       = "clinical_trial.csv",
  time_col   = "survival_time",
  status_col = "event_indicator",
  covar_cols = c("age", "sex", "treatment")
)
```

18.2 simulate_bd_data() — Generate survival data

Simulates from the Beta-Danish distribution with optional right-censoring.

```
# Complete data
dat1 <- simulate_bd_data(n = 200, a = 1.5, b = 2, c = 3, k = 0.5,
                         seed = 42)

# Right-censored data (~20% censoring)
dat2 <- simulate_bd_data(n = 200, a = 1.5, b = 2, c = 3, k = 0.5,
                         censor_rate = 0.2, seed = 42)
```

18.3 simulate_bd_cure_data() — Generate cure-model data

Simulates data with a known cure fraction from either the mixture or promotion-time cure formulation.

```
dat <- simulate_bd_cure_data(n = 300, type = "promotion",
                             a = 1, b = 2, c = 1.5,
                             gamma = c(0.5, -0.4),
                             seed = 2026)
```

Part V — Walkthroughs

19. A Complete Worked Example

The example below walks through a full analysis of the built-in remission dataset (bladder cancer remission times), from loading the data through final diagnostics.

Step 1: Load and inspect the data

```
library(BetaDanish)
data(remission)
head(remission)
summary(remission)
```

Step 2: Plot the empirical Kaplan-Meier

```
km <- survival::survfit(survival::Surv(time, status) ~ 1,
                        data = remission)
plot(km, mark.time = TRUE,
     xlab = "Time (months)", ylab = "Survival",
     main = "Kaplan-Meier estimate")
```

Step 3: Fit the four-parameter Beta-Danish model

```
fit_full <- fit_betadanish(
  survival::Surv(time, status) ~ 1,
  data = remission,
  n_starts = 10
)
summary(fit_full)
```

Step 4: Fit the three-parameter ED submodel

```
fit_sub <- fit_betadanish(
  survival::Surv(time, status) ~ 1,
  data = remission,
  submodel = TRUE,
  n_starts = 10
)
summary(fit_sub)
```

Step 5: Likelihood ratio test (full vs submodel)

```
compare_models(fit_full, fit_sub)
```

Step 6: Compare against standard distributions

```
compare_distributions(fit_full)
```

Step 7: Diagnostic plots

```
plot(fit_full, type = "all")
```

Step 8: Optional Bayesian inference

```
fit_bayes <- bayes_betadanish(
  time     = remission$time,
  status   = remission$status,
  submodel = TRUE,
  burnin   = 2000, mcmc = 5000, seed = 1
)
print(fit_bayes)
```

20. Quick Reference Card

Common tasks and the command to perform them:

Function	Purpose
Install package	install.packages("BetaDanish")
Load package	library(BetaDanish)
Load a built-in dataset	data(remission)
Fit full model	fit <- fit_betadanish(Surv(time, status) ~ 1, data = d)
Fit ED submodel	fit_betadanish(..., submodel = TRUE)
Summarize a fit	summary(fit)
All diagnostic plots	plot(fit, type = "all")
LRT (full vs submodel)	compare_models(fit_full, fit_sub)
Compare to Weibull etc.	compare_distributions(fit)
Goodness-of-fit	gof_betadanish(fit)
Bayesian fit	bayes_betadanish(time, status, submodel = TRUE)
AFT regression	fit_bd_aft(Surv(time, status) ~ age + sex, data = d)
Mixture cure	fit_bd_cure(Surv(t,s) ~ 1, ~ z, data = d, type = "mixture")
Competing risks	fit_bd_competing(time, cause)
CIF vs Aalen-Johansen	cif_compare(fit_cr, plot = TRUE)
Simulate data	simulate_bd_data(n = 200, a = 1.5, b = 2, c = 3, k = 0.5)
Read CSV data	read_survival_data("data.csv", time_col = "t", status_col = "e")
One-call pipeline	analyze_betadanish("data.csv", "t", "e")

Part VI — Appendices

Appendix A — Getting Help

Several resources are available if you get stuck:

- In-R help pages: ?fit_betadanish, ?bayes_betadanish, etc.
- Package website with full function reference and vignettes: https://bilal-aiou.github.io/BetaDanish/
- Source code and issue tracker: https://github.com/bilal-aiou/BetaDanish
- Five built-in vignettes accessible via browseVignettes("BetaDanish") cover the introduction, brain cancer case study, Bayesian estimation, competing risks, and cure models.

Appendix B — Reporting Bugs and Feature Requests

If you find a bug or have a feature request, please open an issue on GitHub at:

https://github.com/bilal-aiou/BetaDanish/issues

Include the output of sessionInfo() and a minimal reproducible example.

Appendix C — Citation

If you use BetaDanish in published work, please cite the underlying paper:

Ahmad, B., & Danish, M. Y. (2025). The Beta-Danish distribution for lifetime data analysis. Journal of Applied Mathematics, Statistics and Informatics, 21(1). doi:10.2478/jamsi-2025-0010

A formatted BibTeX entry is also available in R:

```
citation("BetaDanish")
```

Appendix D — Session Information

When asking for help or reporting issues, always include the output of sessionInfo() so that maintainers can reproduce your environment:

```
sessionInfo()
```

Appendix E — License

BetaDanish is released under the GNU General Public License version 3 (GPL-3). You are free to use, modify, and redistribute the software under the terms of that license. See the file LICENSE in the package source for the full text.

Appendix F — Authors and Acknowledgements

Bilal Ahmad (author and maintainer) — Allama Iqbal Open University, Islamabad, Pakistan. Email: bilalahmad.imcbh9@gmail.com

Muhammad Yameen Danish (author and PhD supervisor) — Allama Iqbal Open University, Islamabad, Pakistan. Email: yameen.danish@aiou.edu.pk

The Beta-Danish distribution was developed as part of the doctoral research of Bilal Ahmad under the supervision of Dr. Danish, and is described in detail in the underlying thesis and the accompanying journal article cited above.